

# Sidrat Marketplace

## Technical document

This PDF captures the current frontend-only implementation of the Sidrat marketplace, including architecture, data model, pages, business rules, and API surface inferred from the codebase.

Generated for the current workspace snapshot.

# 1. Project overview

Sidrat is a frontend-first marketplace prototype centered on group buying mechanics. The app behaves like a fully usable storefront, but persistence and most business state live in local storage rather than a real backend.

- Target users: guests, buyers, sellers, and admins.
- Core value proposition: product discovery, shopping cart flow, group-buying sessions, and an internal wallet model.
- Important constraint: the repo currently does not expose a real production API; many flows are local-shell logic backed by `localStorage` and demo seeding.

## Business goal

The product aims to simulate a marketplace where sellers can publish products, admins can moderate and manage data, and buyers can participate in group-buying sessions with tiered pricing.

# 2. Architecture and tech stack

- Frontend: React + TypeScript.
- Build tool: Vite.
- Routing: react-router-dom.
- State management: Zustand with persist middleware for the cart and related UI state.
- Styling: Tailwind CSS with shadcn/ui components.
- Animation: framer-motion in selected screens.
- Icons: lucide-react.
- Persistence: localStorage for MVP data, app schema versioning, deterministic demo seeding.
- Deployment target: Vercel, with SPA fallback rewrites.

The app is organized as a thin UI shell around a local data layer. The `src/lib/mvp.ts` module acts as the pseudo-backend: it seeds demo entities, normalizes product media, formats currency, and exposes read/write helpers for orders, wallet operations, and sessions.

# 3. Data model

There is no real database in the current implementation. The effective data model lives in TypeScript types and in localStorage collections such as products, sessions, orders, users, wallets, withdrawals, and cart state.

## Main entities

- Product: id, slug, name, category, description, basePrice, media, variants, metadata, seller-facing fields.
- Product variant: id, size, price delta or price, image, stock and optionally sale metadata.
- User: id, role, profile information, wallet balance, and account flags.
- Cart item: product id, quantity, selected size id/label, variant identity, and snapshot pricing.
- Session: group-buying room attached to a product family with slot count, time window, and pricing tiers.
- Order: placed cart checkout result with address, delivery, and payment state.
- Withdrawal request: seller/buyer wallet withdrawal with fee and net amount.

## Entity relationships

- A product can have many variants.
- A product can be linked to zero or more sessions.

- A user can own a cart, wallet transactions, orders, and withdrawals.
- An order contains one or more line items derived from cart items.
- A session references exactly one product family but may expose many purchasable variants.

## 4. User roles and permissions

- Guest: browse home, catalog, product detail, and public session pages.
- Buyer: add to cart, place orders, view profile, manage wallet-related actions.
- Seller: edit own products, manage stock/media, create or update product drafts, and work with seller dashboard tools.
- Admin: broader moderation and management rights, including product and session oversight.

The exact permission matrix is currently enforced at UI and local-helper level rather than a hardened server-side ACL layer.

## 5. Page-by-page behavior

### Home

- URL: ``/``.
- Purpose: landing page, hero carousel, feature highlights, and product preview cards.
- Data flow: loads products from local MVP storage and hero media from the local manifest.
- Key interactions: product navigation, open catalog, read group-buying highlights.

The home screen now uses local media paths for both hero banners and product previews, which keeps localhost and Vercel aligned.

### Catalog

- URL: ``/catalog``.
- Purpose: browse product families with filtering and card-based discovery.
- Data flow: product list and media come from normalized stored data.
- Key interactions: open detail page, filter by category, inspect product imagery.

### Product detail

- URL: ``/product/:slug``.
- Purpose: show full product story, image gallery, size selection, and cart actions.
- Logic: uses canonical shoe-size selection when the product belongs to footwear categories.
- Cart behavior: selected size is attached to the cart item so the same product can coexist in different variants.
- Important UI rule: the primary ``Add to cart`` button stays visible, and the in-cart indicator is a separate action.

### Cart

- URL: ``/cart``.
- Purpose: edit quantities, remove items, and continue to checkout.
- Data flow: reads cart state from Zustand persist storage.
- Media: uses the same local-image helpers as product detail and catalog.

### Checkout

- URL: ``/checkout/:orderId``.
- Purpose: finalize delivery details and confirm the order.
- Behavior: uses city suggestions, address suggestions, and phone normalization helpers.
- External calls: Yandex Suggest API and Russia-cities data file from a CDN.

## Profile

- URL: `/profile`.
- Purpose: user account overview, wallet, orders, and role-specific links.

## Seller dashboard

- URL: `/seller`.
- Purpose: edit products, manage variants, and pick local assets for media fields.
- Media UX: image selectors are driven by the local image manifest instead of free text URLs.

## Admin dashboard

- URL: `/admin/dashboard`.
- Purpose: monitor sessions, sales, and platform metrics.

## Session pages

- URL: `/sessions` and `/sessions/:id`.
- Purpose: browse and inspect group-buying rooms.
- Data flow: session cards and detail views resolve product media through shared helpers.

# 6. Business logic and core mechanics

## Group Buying 2.0

The group-buying model is a tiered pricing system. As slots fill, the effective price can fall according to predefined thresholds. The UI highlights slot count, family linkage, and pricing tiers so the buyer can understand how the deal evolves.

- A session belongs to a product family.
- Slots represent participant capacity.
- Price changes as filled slots increase.
- A ceiling and floor are enforced so the discount stays bounded.

## Wallet logic

The wallet subsystem tracks profit or rebate-like balance updates and withdrawal requests. When a withdrawal is created, the system computes fee and net amount, then stores the request locally.

## Cart identity

Cart items are variant-aware. The combination of product id and selected size/variant identity defines uniqueness, which prevents the selected shoe size from being lost after adding to cart.

## Storage versioning

Local state uses explicit schema versioning so old or corrupted demo data can be dropped and reseeded cleanly. This is what keeps localhost and deployed Vercel data from drifting apart too much.

# 7. API surface

The current repository does not expose a live backend, but the code references the following logical endpoints or external data sources.

- GET `/api/sessions/:sessionId` - planned session detail endpoint.
- GET `/api/sessions/:sessionId/cart/items` - planned cart read endpoint.
- POST `/api/sessions/:sessionId/cart/items` - planned cart add endpoint.
- PUT `/api/sessions/:sessionId/cart/items/:itemId` - planned cart update endpoint.

- DELETE `/api/sessions/:sessionId/cart/items/:itemId` - planned cart delete endpoint.
- GET Yandex Suggest API - address suggestions used in checkout.
- GET Russia cities dataset - city list used by checkout helpers.

## 8. Conclusion

Sidrat is currently best understood as a local MVP shell that already behaves like a real marketplace on the frontend. The strongest parts of the codebase are the normalized media pipeline, schema-versioned persistence, and the tighter cart/size model. The main missing piece is a real backend that would replace the local data layer and make the API surface explicit.

Recommended next step: extract the local MVP logic into a documented service boundary so the app can later swap from localStorage to a real server without rewriting the UI again.